Data Mining Project Report
Rap Italian Songs

Emiliano Sescu (mat. 596883)
Jaanis Fehling (mat. 708851)
Dieudonne Iyabivuze (mat. 703858)

January 5, 2026

https://github.com/jaanisfehling/italian-rap-songs

Contents

1 Data Understanding and Preparation **3**
1.1 Data Understanding . . . 3
1.1.1 Semantic findings inside dataset . . . 4
1.1.2 Missing value imputation . . . 6
1.1.3 Distribution of the variables and statistics . . . 8
1.1.4 Pairwise correlation . . . 8
1.2 Data Preparation . . . 9
1.2.1 Lyrical features . . . 9
1.2.2 Audio features . . . 10
1.2.3 Metadata features . . . 10
1.2.4 Distribution of the variables and statistics . . . 11
1.2.5 Pairwise correlation . . . 11

2 Clustering analysis **13**
2.1 Data Preparation . . . 13
2.2 K-Means Clustering . . . 13
2.2.1 Choosing the Evaluation Criteria . . . 13
2.2.2 Evaluation . . . 13
2.3 Density-based Clustering . . . 15
2.4 Hierarchical Clustering . . . 16
2.5 Further Clustering Methods . . . 18

3 Predictive Analysis & XAI **19**
3.1 Data Preparation . . . 19
3.1.1 Macro–Zones Mapping . . . 19
3.1.2 TF–IDF . . . 20
3.1.3 Feature Selection . . . 20
3.2 Model Training and Hyperparameter Tuning . . . 20
3.2.1 Grid Search and Cross-Validation . . . 20
3.2.2 Random Forest . . . 20
3.2.3 XGBoost . . . 20
3.2.4 LightGBM . . . 21
3.2.5 Support Vector Machine (Linear) . . . 21
3.3 Model Evaluation . . . 21
3.3.1 Test Performance Comparison . . . 22
3.3.2 Early Stopping and Refit Experiment . . . 22
3.3.3 Best Model Analysis . . . 22
3.4 Explainable AI with SHAP . . . 23
3.4.1 Global explanations (feature importance) . . . 23

4 Time series analysis **24**
4.1 Preprocessing . . . 24
4.2 Clustering . . . 24
4.3 Motif Extraction . . . 25
4.4 Anomaly Detection . . . 25
4.5 Shapelets . . . 26

5 Ethical and legal implications **27**
5.1 Ethical Implications . . . 27
5.1.1 Cultural Complexity Reduction and Stereotyping . . . 27
5.1.2 Privacy and Purpose Limitation . . . 27
5.1.3 Gender Bias and Representation . . . 27
5.2 Legal Implications . . . 28
5.2.1 Data Protection and GDPR Compliance . . . 28
5.2.2 Copyright and Text and Data Mining . . . 28
5.3 Mitigation Strategies . . . 28

6 Appendix **30**
6.0.1 Local explanations (single prediction) . . . 30

Chapter 1

Data Understanding and Preparation

This chapter describes how we audited, cleaned, and prepared the dataset for data mining. Before diving into the methodology, it is worth noting that we performed some initial, unstructured exploration to get familiar with the data and the domain. This preliminary phase is not reported here nor included in the notebooks. Instead, we focussed on the actual process we followed, tools we used, and decisions we made. In addition to the notebooks, dedicated Python scripts were developed to update the datasets and generate a final clean version. Consequently, we avoid unnecessary low-level code details in this text, referring the reader to the accompanying notebooks and scripts for implementation specifics.

1.1 Data Understanding

The phase of the dataset exploration focused on assessing the quality of the provided data sources: the *tracks* dataset (11,166 entries, 45 features) and the *artists* dataset (104 entries, 14 features). A preliminary inspection was conducted to identify data type discrepancies, the distribution of missing values (Figure 1.1), and potential semantic anomalies.

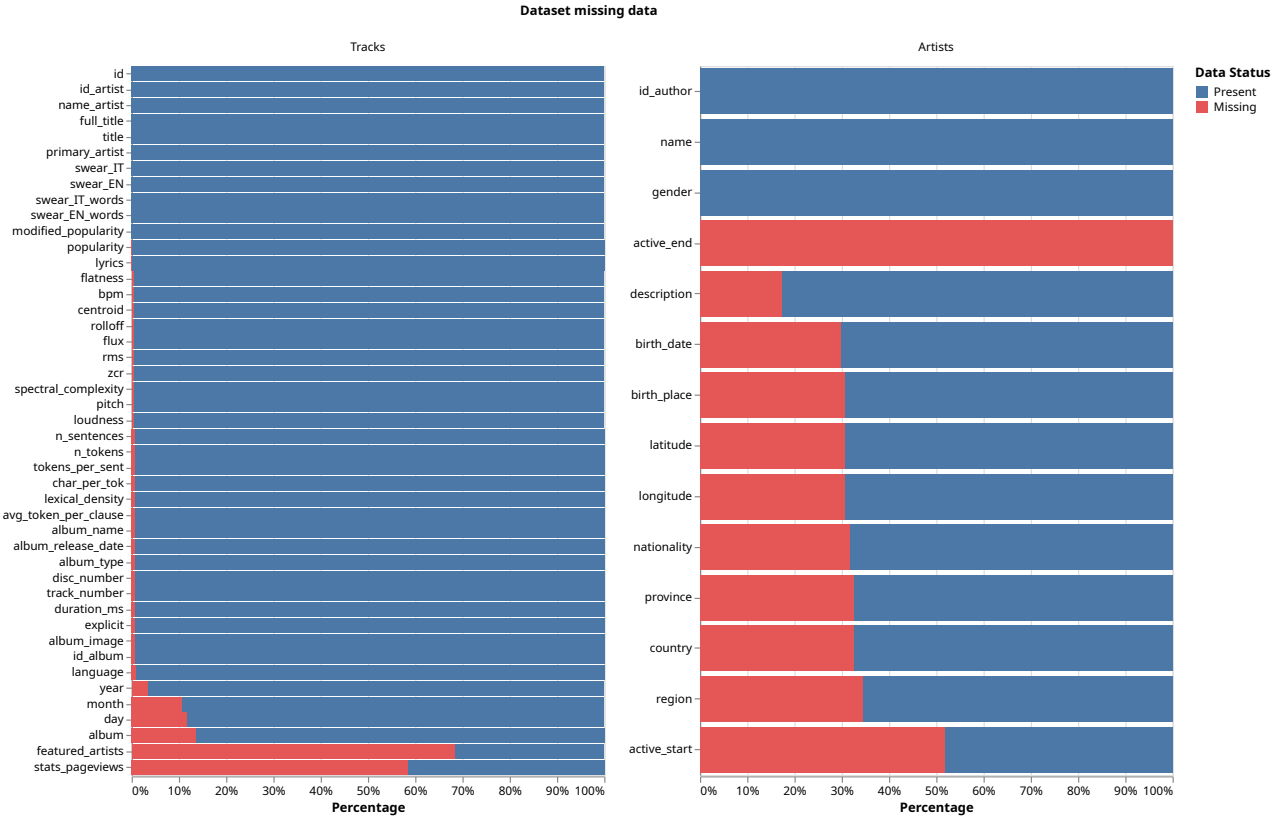

Figure 1.1: Missing data in percentage of tracks and artists datasets

Data type checks (inconsistencies between the stored data types and the expected domain types):

- Inspection of the *tracks* dataframe via `df.info()` revealed that the `year` and `popularity` columns were interpreted as objects, despite representing numerical values. Similarly, `explicit` appeared as an object type requiring conversion to boolean. Other fields semantically representing integers, such as `month`, `day`, `n_sentence`, and `n_tokens`, were interpreted as floats.
- In the *artists* dataset, `birth_date` were interpreted as objects. Upon inspection, this was due to the presence of strings (URL) mixed with valid dates.
- String columns across both datasets were found to contain invisible Unicode characters (e.g., non-breaking spaces, zero-width joiners), which were stripped to ensure proper analysis.

Missing data distribution The analysis of missing values revealed distinct patterns of sparsity:

- Tracks: High missing data percentage was observed in `featured_artists` (68%), which is expected as not all tracks feature collaborations. `stats_pageviews` was missing for approximately 58% of records, likely due to incomplete data from the scraping source and variability in release dates. While core metadata like `id`, `title`, `name_artist`, etc. are complete for all the tracks, audio features extracted with signal processing (`bpm`, `loudness`, `rolloff`, etc.) are missing for 64 of the tracks (approximately 0.6%).
- Artists: The `active_end` column was entirely null (100% missing). `province`, `region`, `latitude` and `longitude` were missing in approximately 30% of cases, limiting the usability of demographic features without imputation.

1.1.1 Semantic findings inside dataset

Following the broad assessment of missing values, a deeper semantic analysis was conducted to validate the integrity of the data fields and their relationships.

- Duplicate and colliding IDs: The tracks `id` field in the tracks dataset, expected to be unique, contained 71 non-unique identifiers. Detailed inspection revealed that 70 of these were collisions where different songs shared the same ID. One case (`IDTR367132`) was referring to 4 records, with 2 of them being unique tracks.

  Analysis of `id_album` revealed 3,061 unique IDs with significant mapping inconsistencies. While most IDs map 1:1, 5 IDs map to conflicting album names. Conversely, 337 album names are associated with multiple distinct IDs. These discrepancies suggest imputation errors rather than genuine distinct releases; consequently, these records are not considered reliable for subsequent imputation steps (see year imputation).

- Lyrics: The `lyrics` column contained non-lyrical content in some of the records, spotted by two distinct patterns. The first involved tracks matching the format "{n} Contributor/s{title}". Within this pattern, we distinguished a subset of 13 valid but malformed tracks containing the sequence "{n} Contributor/s{title} Lyrics{lyrics}", which were retained despite formatting issues (in those cases, lyrics were stored in a single line); the remainder were classified as instrumentals or metadata artifacts. The second case concerned tracks starting with "La produzione...". These were validated using the `n_sentences` metric: records with a single sentence were confirmed as instrumentals.

- Language Classification: We identified significant mislabeling in the `language` feature. A manual inspection revealed that many tracks containing Italian lyrics were incorrectly assigned to other languages. This was most obvious with uncommon codes like `da` (Danish), `cs` (Czech), and specifically `pl` (Polish), where nearly 600 Italian tracks were misclassified. However, even common labels like `en` (English) and `es` (Spanish) were found to erroneously contain Italian tracks.

- Swear words set: An examination of the swear word set revealed swearwords misslanguaged and words that were not swearwords. We also identified that the detection logic often missed grammatical variations of known Italian swear words. Specifically, while the root forms were present, singular, plural, and gendered variations were frequently absent, leading to inaccurate counts in `swear_IT` and `swear_IT_words`.
- Artist name variations: A cross-reference between the *tracks* and *artists* datasets revealed 10 mismatches in artist names (related to name formatting, abbreviation and pseudonym).
- Errors and semantically incorrect data: Data type issues come mostly from error values inside the features. The *artists* dataset contained URLs in the `birth_date` column. In the *tracks* dataset, we found release years dated in both the future and the past, as well as strings inside `popularity` values or containing numbers outside it's domain (0–100 range). Other error values are better described in the notebooks.
- Useless and redundant fields: The `active_end` feature in the artists dataset is empty. In the tracks dataset `primary_artist` and `name_artist` contain the same values. Additionally, both `album` and `album_name` appeared to contain (different versions of) the albums title.

This process uncovered specific anomalies requiring intervention:

- Duplicate and colliding IDs: We resolved duplicate `id` entries by identifying the highest existing numeric index (format: `TR{unique_num_id}`) and assigning conflicting records new IDs generated by incrementally increasing this maximum value. Same strategy was used for `id_album`.
- Language classification: `langdetect`[1] was employed to correct these misclassifications and fill missing values. Additionally, the code `co` (Corsican) was identified as a proxy for Italian and merged accordingly. The missing language values are related to the tracks with no lyrics.

Table 1.1: Final tracks language distribution

Language	**Track Count**
it	10,828
en	147
(missing)	128
es	40
pt	6
fr	5
de	2
ru	1
bg	1

- Lyrics: A total of 126 tracks with no real lyrics were found using regular expressions on the patterns "{n} Contributor/s{title}" and "La produzione...", with the subsequently nullification of lyrics feature and their related linguistic features.
- Swear words set: We upgraded the set of Italian swear words to ensure semantic completeness. This involved removing manually terms incorrectly flagged as offensive and systematically adding the missing singular, plural, and gendered forms of existing swear words. Following this refinement, the values for the columns `swear_IT`, `swear_EN`, `swear_IT_words` and `swear_EN_words` were recomputed, and the `explicit` binary feature was updated accordingly.
- Artist name variations: The `name` column in the *artists* dataset is the one with the better variations, as it consistently provided a more normalized and complete artist's name (e.g., full names like *chadia rodriguez* and *guè pequeno* versus the abbreviated *Chadia* and *Guè*)

[1] https://pypi.org/project/langdetect/

- Errors and semantically incorrect data: In the *artists* dataset, the `birth_date` column contained a URL instead of a timestamp, which was flagged for cleaning.
- Useless and redundant fields: The `active_end` feature in the *artists* dataset and the `name_artist` feature in the *tracks* dataset were removed. Structural analysis indicated that `id_album` and auxiliary columns (e.g., `album_image`) consistently referenced the content found in `album_name`, identifying it as the correct album name column over `album`.

1.1.2 Missing value imputation

The dataset presented several missing values across both artist and track features. To ensure data integrity, a mixed-strategy approach was employed for imputation, categorized by the source of the information used: internal derivation, external scraping, and model-based generation.

Internal Data Derivation

Certain missing fields were filled by leveraging logical relationships between existing data points within the dataset (see the complete list inside the jupyter notebooks).

- Artist activity: 54 artists had missing `active_start` dates. These were imputed by cross-referencing the tracks dataset to identify the earliest release date associated with each artist.
- Album release dates: For albums with conflicting or multiple release dates, the earliest date was selected as the canonical release date.
- Track IDs: Duplicate track IDs referring to distinct songs were resolved by generating new, unique identifiers following the already explained incremental format.
- Character metrics: Missing values for `char_per_tok` (characters per token) were structurally imputed by calculating the ratio of non-whitespace characters to the token count derived from the lyrics.

External Sources (Scraping and APIs)

When internal data was insufficient, external APIs were utilized to retrieve missing information.

- Artists geographic data: To impute missing province and region information, a multi-stage approach was adopted. Initially, the `Nominatim API`[2] (OpenStreetMap) was used to reverse-geocode entries that possessed latitude and longitude coordinates but missed `region` and `province`. Then `WikiData`[3] was queried to retrieve data for the remaining artists. This scraping process was performed iteratively: queries were first executed using names from the *artists* dataset, followed by names from the *tracks* dataset, as the latter often provided different (and better) formatting. The few remaining entries that could not be resolved via these automated methods were manually input to ensure dataset completeness.
- Tracks release date The `Spotify API`[4] was utilized as the primary source to retrieve exact release dates for individual songs (Figure 1.2 shows `year` distribution before and after imputation). For tracks where the API failed to return a specific date or the query was unsuccessful, the missing values were imputed by defaulting to the release date of the corresponding album.

[2]https://nominatim.org/release-docs/latest/api/Overview/
[3]https://www.wikidata.org/wiki/Wikidata:Main
[4]https://developer.spotify.com/documentation/web-api

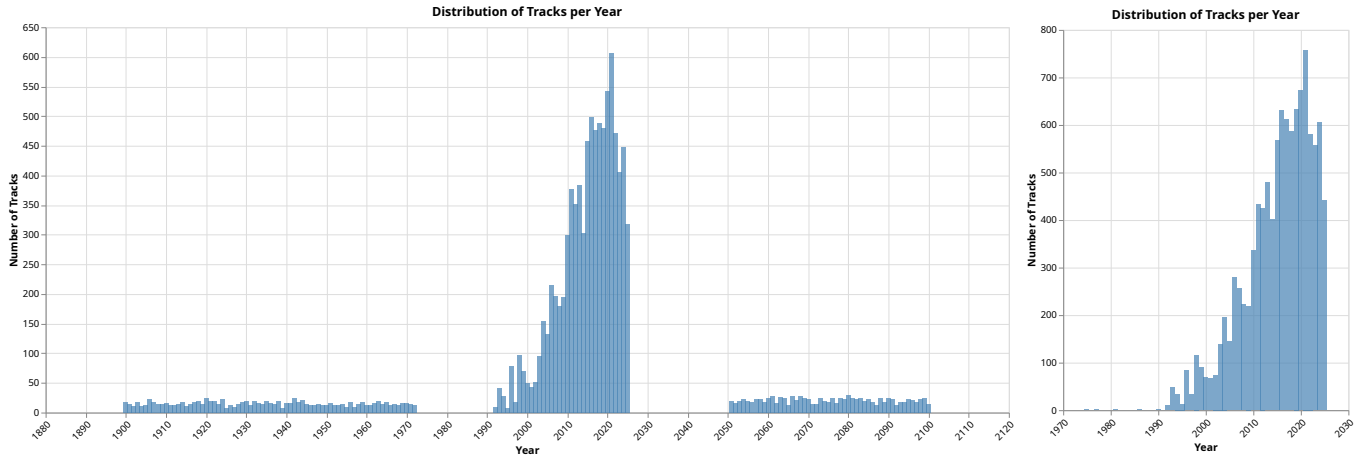

Figure 1.2: Distribution of tracks per year before and after errors elimination + imputation

Model-Based and Algorithmic Imputation

For complex text-based features, algorithmic models were applied.

- **Language Detection:** We identified 203 tracks with missing `language` labels; 128 were instrumental, and the rest were classified using the `langdetect` library. Additionally, we corrected mislabeled Italian tracks that were erroneously coded as non-Italian. We re-evaluated these tracks using `langdetect`, updating the labels to 'it' or the appropriate language code based on the detection output. Anomalous language results underwent manual verification.

- **NLP Feature Imputation (Regex vs. SpaCy):** To impute missing values for NLP-derived features, we first performed a *comparative validation analysis* on the existing non-missing data. We attempted to reproduce the original values using two distinct computation methods:

  1. `SpaCy:`[5] Advanced linguistic tokenization and lemmatization (lightweight models for italian and english).
  2. `Regex:` Simple regular expression-based patterns (counting newlines or whitespace-separated strings - explore the patterns in the notebooks).

  We compared the re-computed values against the original dataset's ground truth, prioritizing the method that maximized *exact row matches* and minimized *mean difference* and *variance*. This reverse-engineering process determined the optimal imputation strategy for each feature:

  – Sentence Count (`n_sentences`): The analysis revealed that the original dataset likely defined a "sentence" as a new line rather than a grammatical unit. Regex achieved 4,767 exact matches (and mismatches were differing by a small count) compared to only 95 for SpaCy. Consequently, Regex was selected as the primary imputation method, with SpaCy used only for lyrics discovered to be contained in a single line.

  – Token Count (`n_tokens`): Regex outperformed SpaCy in alignment with the original data logic (590 vs. 340 exact matches). Furthermore, SpaCy showed a significant negative mean difference (-8.40), indicating it consistently undercounted tokens relative to the dataset's original definition. Regex provided a closer approximation (mean difference: $+3.49$)

  – Lexical Density (`lexical_density`): While both methods produced similar statistical errors (standard deviation ≈ 0.116 for both), Regex was chosen because it produced nearly three times as many exact matches (213 vs. 76) and offered better computational efficiency.

[5] https://spacy.io/

– Characters per Token (`char_per_tok`): Validation attempts showed that simple Regex counting (excluding all whitespace) achieved only 45 exact matches, suggesting the original data had inconsistent punctuation handling (as already proven in `n_tokens`). To ensure consistency in imputation, we calculated this value using a standardized formula: $\frac{\text{total non-whitespace characters}}{\texttt{n\_tokens}}$.
– Clauses (`avg_token_per_clause`): Since grammatical clauses cannot be reliably identified via Regex patterns, SpaCy was used exclusively to impute this feature.

1.1.3 Distribution of the variables and statistics

To assess the quality of the numerical features, we analyzed their distributions using histograms (Figure 1.3) and box plots. By combining this visual inspection with semantic domain knowledge, we established specific thresholds to distinguish between natural statistical outliers and actual data errors.

The analysis identified three erroneous tracks for removal: one with an impossible BPM of over 700 (likely an extraction error) and two displaying almost zero values across multiple spectral features (*centroid*, *rolloff*, *flux*, *spectral complexity*, *rms*, *zcr*, ...), suggesting they were processed from damaged audio files.

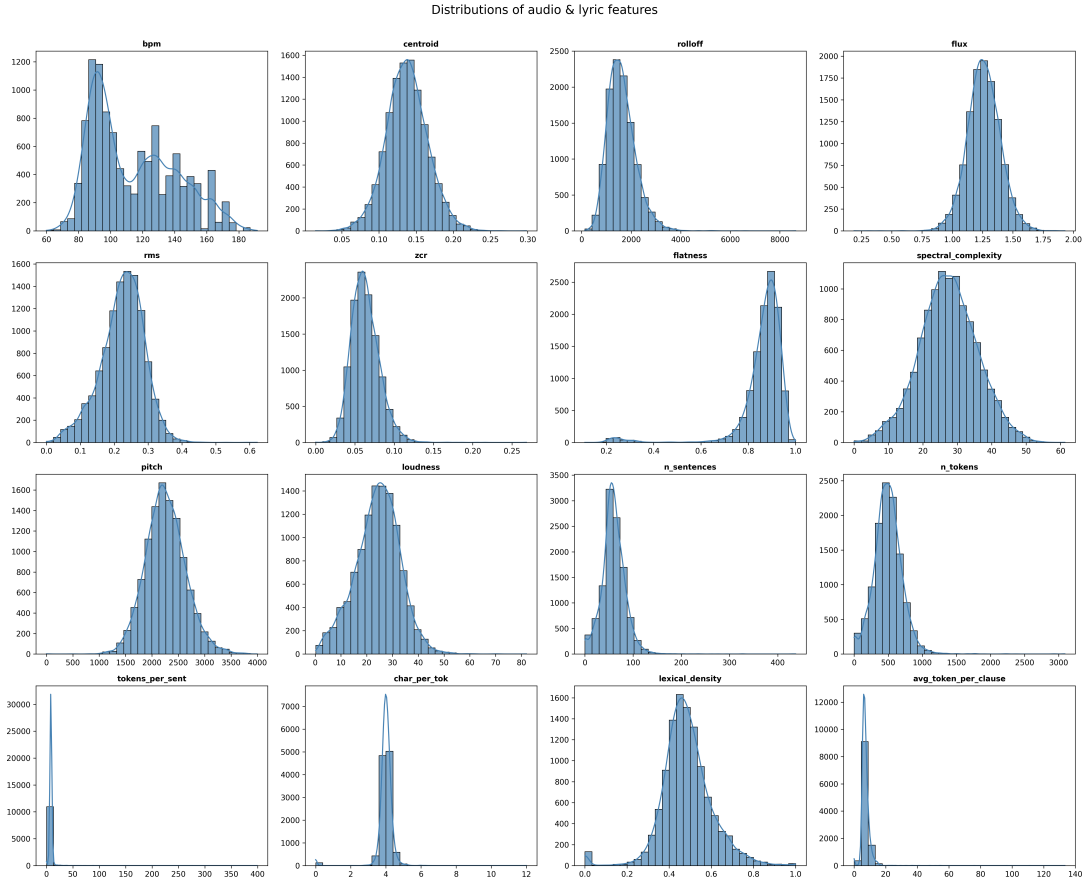

Figure 1.3: Histograms showing distribution of some numerical features.

1.1.4 Pairwise correlation

To understand the relationships between the features in the dataset, a Pearson correlation analysis was performed on all numerical variables. The full correlation matrix is visualized in Figure 1.4. This analysis revealed strong redundancies between certain spectral and dynamic features, as well as structural relationships within the lyrical data. Based on these findings, specific variables were

excluded to reduce multicollinearity, while others were intentionally retained to preserve semantic distinction. The key correlation pairs and the rationale for their selection are detailed below:

- **High Correlation (> 0.9):**
  - `loudness` vs. `rms` ($\rho = 1$): Since `loudness` is effectively computed from `rms` but scaled to decibels, it was retained as the better feature. The decibel scale aligns more closely with human auditory perception compared to the raw linear energy measured by `rms`, which was consequently excluded.
  - `rolloff` vs. `zcr` ($\rho = 0.97$): We prioritized `rolloff` because it identifies the specific frequency below which a majority of the total spectral energy lies. In contrast, `zcr` is defined simply as the rate at which the signal changes sign. As `zcr` is significantly more sensitive to noise artifacts than the spectral approach, it was excluded in favor of `rolloff`.
- **Medium Correlation ($0.7 < |\rho| \leq 0.9$):**
  - `n_tokens` vs. `n_sentences` ($\rho = 0.87$): Despite the high correlation, both metrics were retained to capture the full structural magnitude of the lyrics. `n_tokens` represents the total verbal volume, while `n_sentences` represents the structural segmentation of the lyrics.
  - `centroid` vs. `rolloff` ($\rho = 0.78$): While both features relate to the "brightness" of the sound, they describe different spectral shapes. The `centroid` represents the spectral "center of mass", indicating the average dominant frequency, whereas `rolloff` marks the boundary frequency below which the majority of energy lies. Retaining both enables the model to distinguish between tracks with a high average pitch (high centroid) and those with a broad frequency range (high rolloff).

1.2 Data Preparation

In this phase, several new features were engineered to capture lyrical sophistication, sonic characteristics, and contextual metadata. These features were designed to distinguish specific sub-genres and production styles.

1.2.1 Lyrical features

Words Per Minute (WPM) Represents the delivery speed of the artist, distinguishing fast flows from slower, melodic styles.

$$WPM = \frac{N_{tokens}}{T_{min}} \tag{1.1}$$

Syllables Per Beat (SPB) Measures the rhythmic density of the flow relative to the instrumental beat rather than absolute time.

$$SPB = \frac{N_{tokens}}{T_{min} \times BPM} \tag{1.2}$$

Explicitness Density ($D_{explicit}$) The ratio of explicit terms (in both Italian and English) to the total number of words, indicating the swear words intensity of the content.

$$D_{explicit} = \frac{S_{IT} + S_{EN}}{N_{tokens}} \tag{1.3}$$

Lyrical Complexity ($C_{lyrical}$) A composite metric weighting vocabulary richness (lexical density) by words lenght (average characters per token).

$$C_{lyrical} = D_{lexical} \times L_{word} \tag{1.4}$$

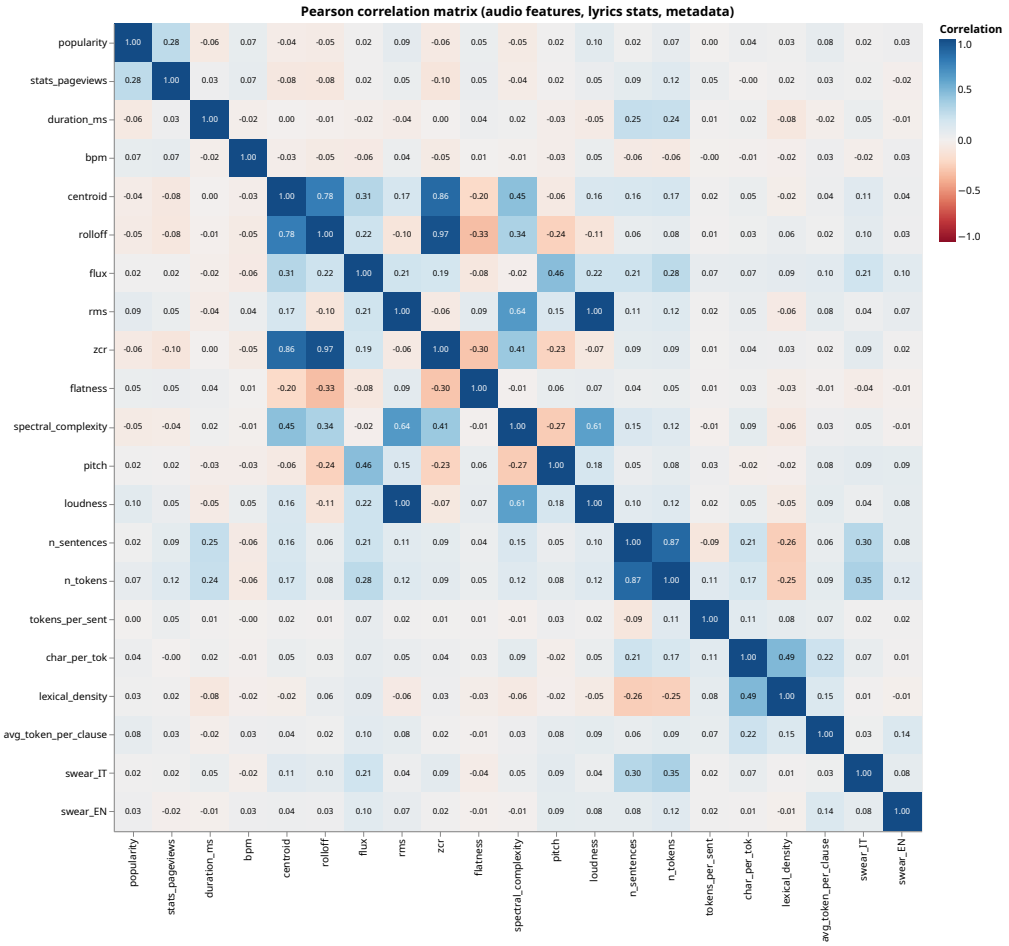

Figure 1.4: Pearson correlation matrix containing original dataset features.

1.2.2 Audio features

Audio Aggressiveness (A_{agg}) A composite score distinguishing high-energy tracks from laid-back productions by combining speed, intensity, and timbral harshness. Features were normalized using Min-Max normalization to account for different scales (Hz, dB, BPM).

$$A_{agg} = \frac{1}{4} \sum_{f \in \{rolloff, loudness, flux, bpm\}} \frac{x_f - \min(x_f)}{\max(x_f) - \min(x_f)} \tag{1.5}$$

Harmonic Complexity ($C_{harmonic}$) Indicates instrumental richness by prioritizing sounds that are spectrally complex (many peaks) but tonal rather than noisy.

$$C_{harmonic} = C_{spectral} \times (1 - F_{flatness}) \tag{1.6}$$

Vocal Clarity ($V_{clarity}$) Estimates the brightness of the mix. High values indicate significant high-frequency presence (rolloff) relative to the overall loudness, typical of modern, crisp vocal productions.

$$V_{clarity} = \frac{R_{rolloff}}{|L_{loudness}|} \tag{1.7}$$

1.2.3 Metadata features

Collaboration Count (N_{collab}) The count of unique featured artists on a track, serving as a proxy for structural variety and vocal texture diversity.

$$N_{collab} = |\{a \mid a \in \text{featured\_artists}\}| \tag{1.8}$$

Artist Relative Popularity (Z_{artist}) Identifies if a track is a "hit" relative to the specific artist's average performance, using a Z-score standardization.

$$Z_{artist} = \frac{P_{track} - \mu_{artist}}{\sigma_{artist}} \tag{1.9}$$

Season Relative Popularity (Z_{season}) Standardizes popularity based on the release season to account for seasonal listening trends.

$$Z_{season} = \frac{P_{track} - \mu_{season}}{\sigma_{season}} \tag{1.10}$$

1.2.4 Distribution of the variables and statistics

Following the creation of the new variables, we applied the same rigorous assessment methodology used for the raw data. To assess the quality of the engineered features, we analyzed their distributions using histograms and box plots (see Figure 1.5). By combining this visual inspection with semantic domain knowledge, we established specific thresholds to distinguish between natural statistical outliers and actual calculation artifacts.

This analysis was crucial for refining the *vocal_clarity* feature. The initial distribution revealed extreme outliers caused by tracks with loudness values near the digital ceiling of 0 dB. These near-zero denominators resulted in asymptotic values that distorted the feature space. To correct this, we imposed a minimum threshold on the denominator (treating $|loudness| < 1$ dB as 1 dB), which stabilized the metric while preserving its semantic intent. The remaining features, such as *words_per_minute* and *audio_aggressiveness*, exhibited consistent distributions within expected semantic ranges.

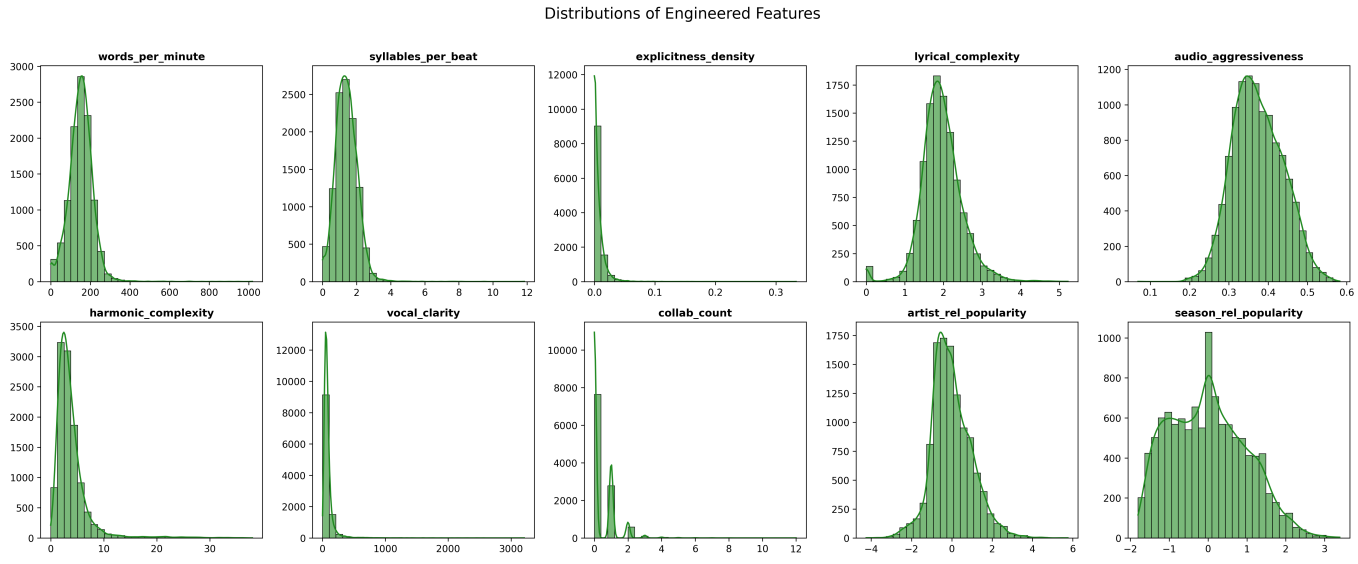

Figure 1.5: Histograms showing distribution of engineered features.

1.2.5 Pairwise correlation

To understand the relationships between the engineered features and the existing dataset, a Pearson correlation analysis was performed. The full correlation matrix is visualized in Figure 1.6. An analysis of the matrix reveal correlation on different features:

- **High Correlation** (> 0.9)**:**
  - `season_rel_popularity` vs `popularity` ($\rho = 0.96$): The engineered feature is almost identical to the raw popularity score, offering little new information. Consequently, `season_rel_popularity` was not used for other analysis.
  - `lyrical_complexity` vs `lyrical_density` ($\rho = 0.92$): Since lyrical complexity is derived directly from lexical density, the strong linear relationship was expected. To reduce redundancy and complexity, `lyrical_complexity` was not used for other analysis.
- **Medium Correlation** ($0.7 < |\rho| \leq 0.9$)**:**
  - `words_per_minute` vs `syllables_per_beat` ($\rho = 0.87$): Although statistically redundant, these features capture distinct dimensions of vocal delivery. `words_per_minute` measures absolute information density over time, while `syllables_per_beat` measures rhythmic density relative to the musical grid. Retaining both allows the clustering algorithm to distinguish between fast-tempo tracks with sparse lyrics and slow-tempo tracks with double-time flows.
  - `harmonic_complexity` vs `flatness` ($\rho = -0.84$): This inverse relationship validates the technical quality of the audio features but does not imply total redundancy. `flatness` is a raw signal processing metric distinguishing noise-like textures from tonal sounds, whereas `harmonic_complexity` measures chordal richness. Keeping both ensures the model can differentiate between harmonically simple tracks and "noisy" tracks, which have similar complexity scores but different spectral profiles.
  - `artist_rel_popularity` vs `popularity` ($\rho = 0.80$): While raw `popularity` indicates a track's current success, `artist_rel_popularity` contextualizes this against the artist's baseline fame. By including both, the clustering model can separate "sleeper hits" (high track popularity, low artist fame) from "expected hits" (high track popularity driven by high artist fame).
  - `audio_aggressiveness` vs `bpm` ($\rho = 0.78$): Tempo is a major component of perceived energy, but it is not the sole driver. `audio_aggressiveness` likely incorporates dynamic range and frequency spread beyond simple speed.

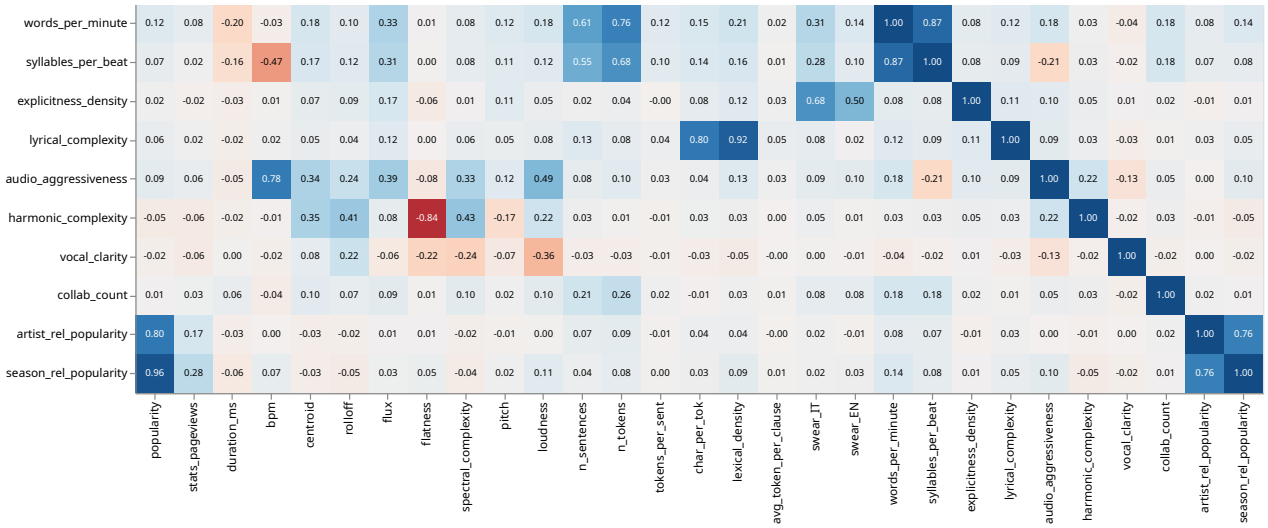

Figure 1.6: Pearson correlation matrix containing engineered features.

Chapter 2

Clustering analysis

2.1 Data Preparation

After loading the dataset that was modified by the previous Data Understanding task, we first remove all non-continuous features from the dataset. The remaining continuous features were standardized by removing the mean and scaling to unit variance. Samples with missing values were removed.

2.2 K-Means Clustering

The evaluation of K-Means and the finding of an optimal value for k, the number of clusters, goes hand in hand. We run the algorithm nine times, for each k between 2 and 10 and evaluate the results using four different evaluation criteria to choose the best possible value of k. To achieve best possible results, instead of using purely random initialization of regular K-Means, the *K-Means++* algorithm [1] is used. Additionally, K-Means is run 100 times for each k and *Sum of Squared Errors* (SSE) is used to determine the best result.

2.2.1 Choosing the Evaluation Criteria

Evaluating a clustering result is not a trivial task, since there is no ground truth available and we cannot rely on accuracy or other metrics from supervised learning. To choose the correct number of clusters k, a common way is to use the *elbow method* [8]: After plotting the clustering results by SSE for each k, the point at which the decrease of error levels off is called the elbow and that point should be used as the final number of clusters k. The elbow criterion is popular especially in educational literature, even tho it is totally subjective has no theoretic support [6]. Even for uniform, random data, the SSE curve follows a slope in which an elbow could fit, furthermore plotting more datapoints on the graph can alter the location of the perceived elbow. Schubert [6] compares different evaluation criteria that can be useful for finding the optimal number of clusters k without having to rely on the elbow criterion. We use the Silhouette Score [5], Variance Ratio Criterion [2] and Davies-Bouldin Index [3] in addition to SSE and instead of elbow use a local minimum or maximum to find k. Another criterion that is often used in literature is the Bayes Information Criterion (BIC) [7], but it cannot be used for K-Means because BIC is derived for probabilistic models and K-Means is not probabilistic by design. Looking at Figure 2.1, we identify $k = 9$ as a local optimum in both the Silhouette Score and Davies-Bouldin Index plot.

2.2.2 Evaluation

The most important method when evaluating K-Means clustering results is looking at the values of each feature of the k centroids, or in other words the average feature values of each cluster. This can be visualized in a table, where each row represents a cluster and each column a feature. In each cell the average feature value of the respective cluster can be found. Another row is added to compare

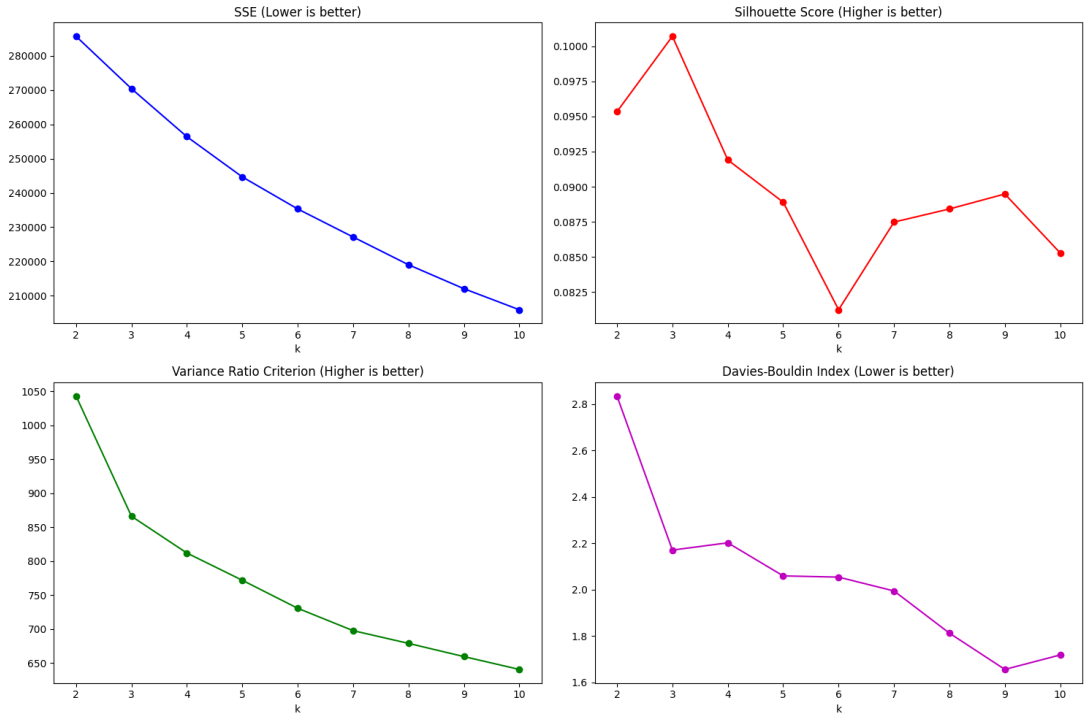

Figure 2.1: Results of running K-Means with different evaluation criteria plotted for each value of k between 2 and 10.

with the average values across the entire dataset. The resulting table for the most important features of the K-Means clustering can be found in Table 2.1.

Table 2.1: Clustering result of the most important features of K-Means clustering. Feature values are within-cluster averages. Year was added even though it had less important in clustering.

Cluster ID	Members	audio_aggressiveness	harmonic_complexity	n_tokens	year
Cluster 0	422	0.40	3.89	651.91	2016.18
Cluster 1	2445	0.34	2.86	657.93	2010.99
Cluster 2	2350	0.42	5.79	477.68	2014.35
Cluster 3	2829	0.43	3.18	537.82	2019.79
Cluster 4	14	0.41	3.23	427.71	2016.93
Cluster 5	2720	0.35	2.29	328.50	2015.84
Cluster 6	243	0.37	20.24	447.95	2009.38
Cluster 7	128	0.34	3.23	0.84	2008.71
Cluster 8	7	0.30	1.26	433.00	2013.71
All Clusters	11158	0.38	3.85	496.44	2015.25

A visual representation of the data in a scatterplot is impossible because of its high dimensionality, but we can make use of *Principal Component Analysis* (PCA) to reduce the dimensionality to three and create a scatterplot with colors indicating cluster assignments. Looking at the plot in Figure 2.2 we can see a uniform blob and some outliers. The low Silhouette Score (≈ 0.9) indicates that points are almost equally distant from their own cluster as from others. The Davies-Bouldin Index above 1 (≈ 1.6) shows that clusters are similar and overlapping. We conclude that the data presents a single, mostly homogeneous distribution (aside from some outliers) and there are no inherent substructures in the data that would allow for a meaningful and theoretically grounded clustering analysis.

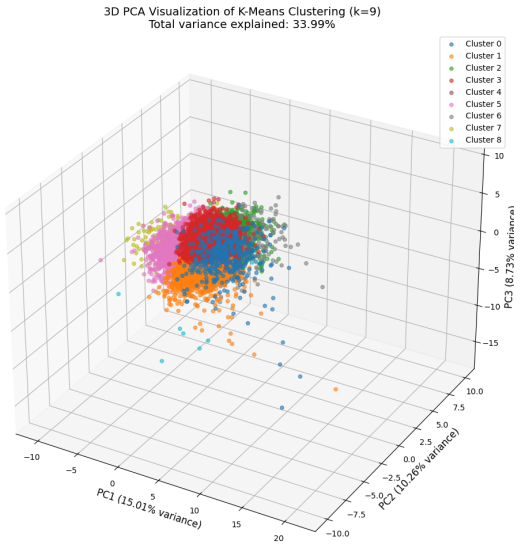

Figure 2.2: First three principal components of the dataset. Colors indicate cluster assignments of K-Means.

2.3 Density-based Clustering

We can evaluate clustering results from DBSCAN the same way as for K-Means; using Silhouette Score, Variance Ratio Criterion and Davies-Bouldin Index. Only SSE is not used since there is no notion of a centroid in DBSCAN clustering. Additionally, we can plot how much noise points we observe for each clustering run. Further, we need to plot how many clusters have been found by DBSCAN. Instead of searching for an optimal k as for K-Means, we try to find the best ϵ. Looking at Figure 2.3, we choose $\epsilon = 4.5$.

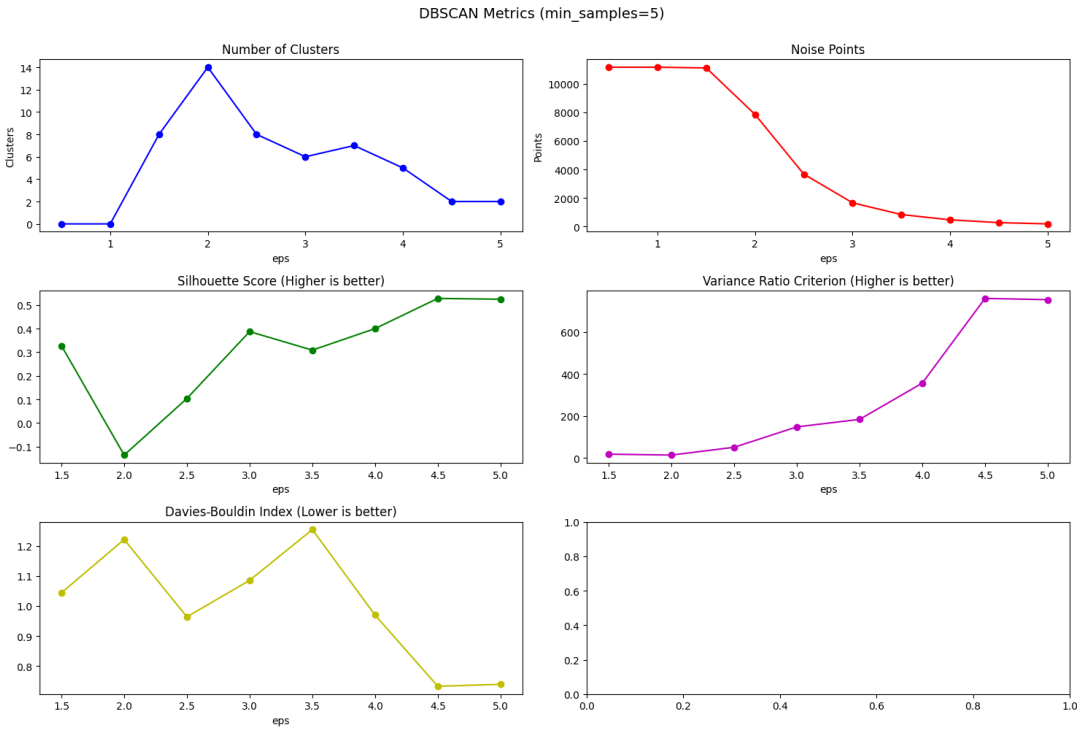

Figure 2.3: Results of running DBSCAN with different evaluation criteria plotted for each value of ϵ between 1 and 5.

Table 2.2 exhibits a clear imbalanced cluster distribution of one big cluster and a smaller one. The plot in Figure 2.4 shows that DBSCAN will not separate clusters in a uniform blob. The high Silhouette Score shows that this is well-separated clustering, which supports our hypothesis

of a (mostly) uniform blob. Looking at Table 2.2 we can also see that the second, smaller cluster contains mostly old songs, showing that atleast there is a small separatable cluster containing older tracks.

Table 2.2: Clustering result of the most important features of DBSCAN clustering.

Cluster ID	Members	lexical_density	n_tokens	tokens_per_sent	year
Noise	273	0.51	593.53	12.86	2012.29
Cluster 0	10765	0.51	499.51	8.59	2015.40
Cluster 1	120	0.00	0.00	0.00	2008.78
All Clusters	11158	0.51	496.44	8.60	2015.25

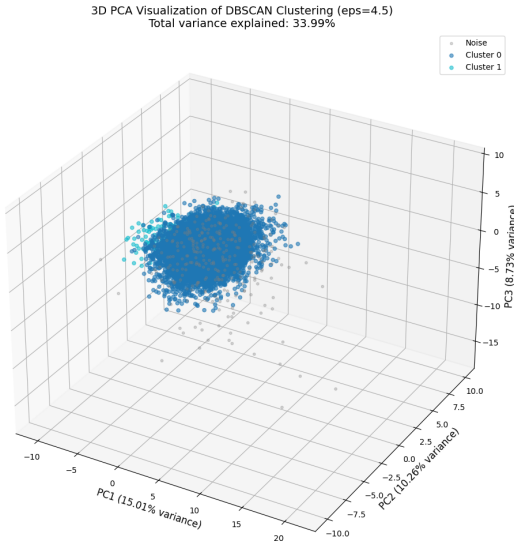

Figure 2.4: First three principal components of the dataset. Colors indicate cluster assignments of DBSCAN.

2.4 Hierarchical Clustering

The dendrograms in Figure 2.5 show different linkage criteria. We choose the ward linkage criteria because it has the most balanced cluster distribution. Like for DBSCAN, we use the same evaluation metrics except for SSE, because again there is no notion of centroids in hierarchical clustering. However, similarly to K-Means, we need to use the evaluation metrics to find an optimal k number of clusters. Figure 2.6 shows the best combination for Silhouette Score and Davies-Bouldin Index for $k = 4$.

The very low silhouette Score and the high Davies-Bouldin Index (2.6) confirm again the uniform blob that can be seen in Figure 2.7. The dendrograms also show that agglomerative clustering has a hard time finding a balanced cluster partition for the data. The analysis of the most important features can be found in Table 2.3.

Table 2.3: Clustering result of the most important features of Agglomerative clustering with ward linkage.

Cluster ID	Members	n_tokens	words_per_minute	audio_aggressiveness	year
Cluster 0	7417	562.64	170.90	0.40	2015.32
Cluster 1	241	451.58	139.12	0.36	2009.21
Cluster 2	3373	372.73	116.35	0.36	2015.74
Cluster 3	127	0.84	1.47	0.34	2008.58
All Clusters	11158	496.44	151.88	0.38	2015.25

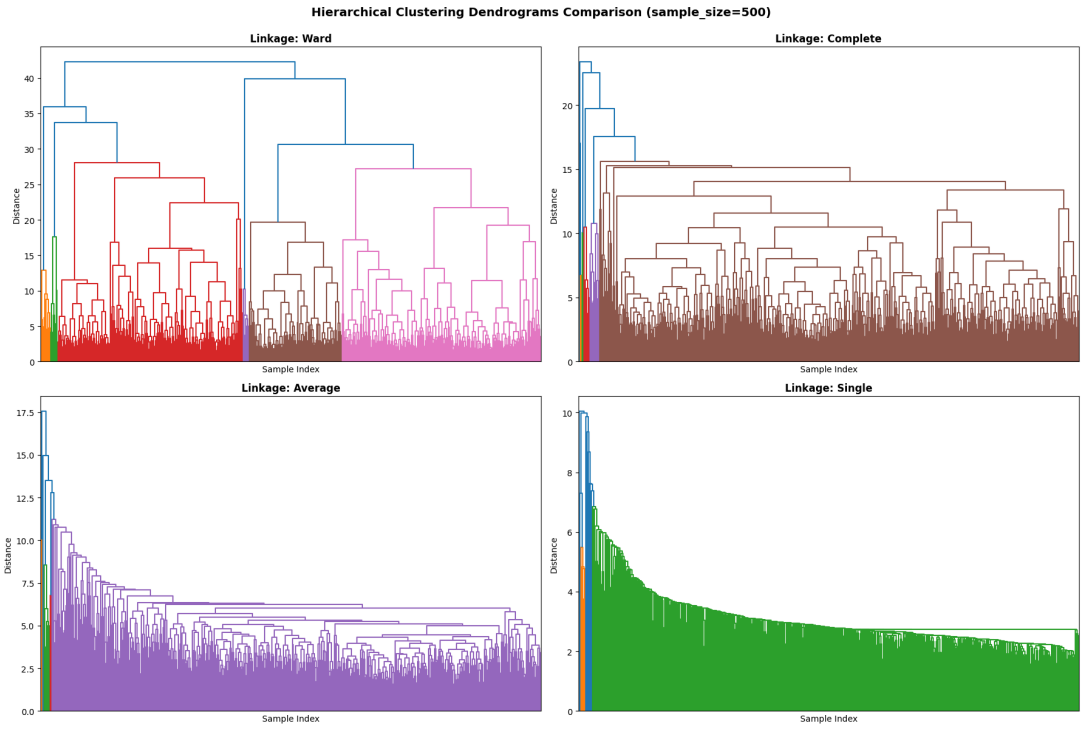

Figure 2.5: Four Dendrograms comparing different linkage methods using euclidean distance. The dataset was subsampled to 500 instances for better visualization.

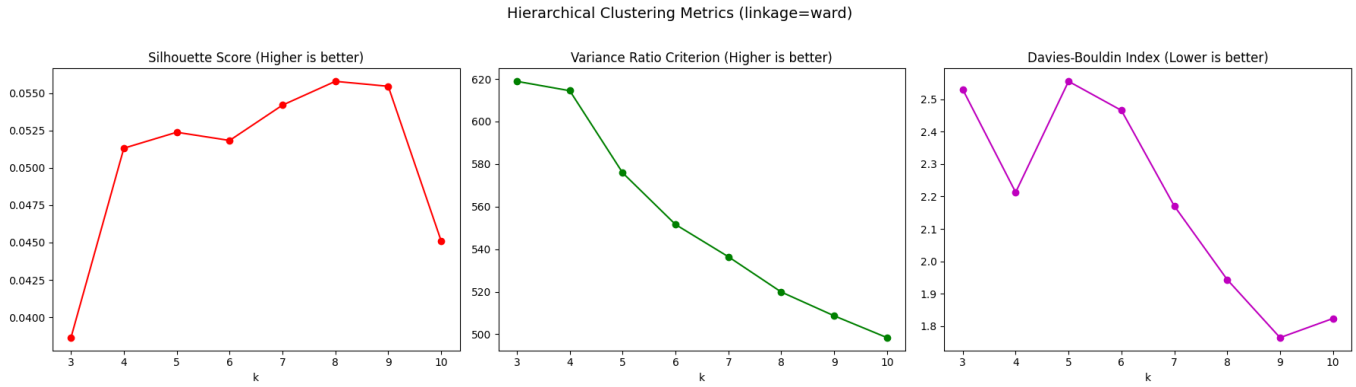

Figure 2.6: Results of running Agglomerative clustering with different evaluation criteria plotted for each value of ϵ between 3 and 10.

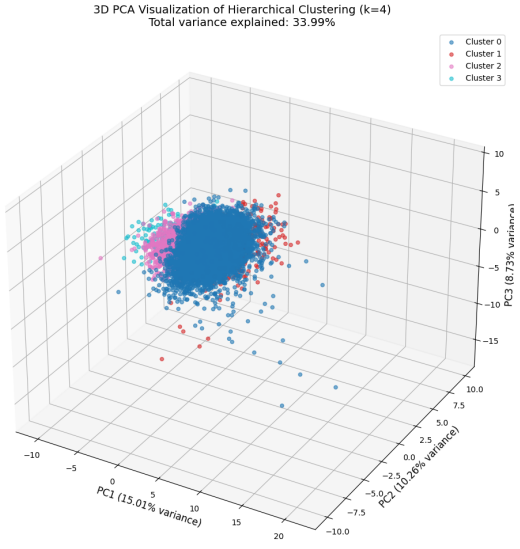

Figure 2.7: First three principal components of the dataset. Colors indicate cluster assignments of Agglomerative clustering.

2.5 Further Clustering Methods

We use X-Means and K-Medoids as alternative clustering methods, both from the pyclustering Python library. The plots in Figure 2.8 show that X-Means ended on $k = 20$ and beat K-Medoids in SSE and Davies-Bouldin Index, but was unable to beat it in Silhouette Score and Variance Ratio Criterion. The overall range of the metrics is similar to K-Means with subtle difference (that fluctuate due to randomness). This also means that the metrics indicate no well separated clustering results and there is no additional analysis we can conclude from these clustering methods.

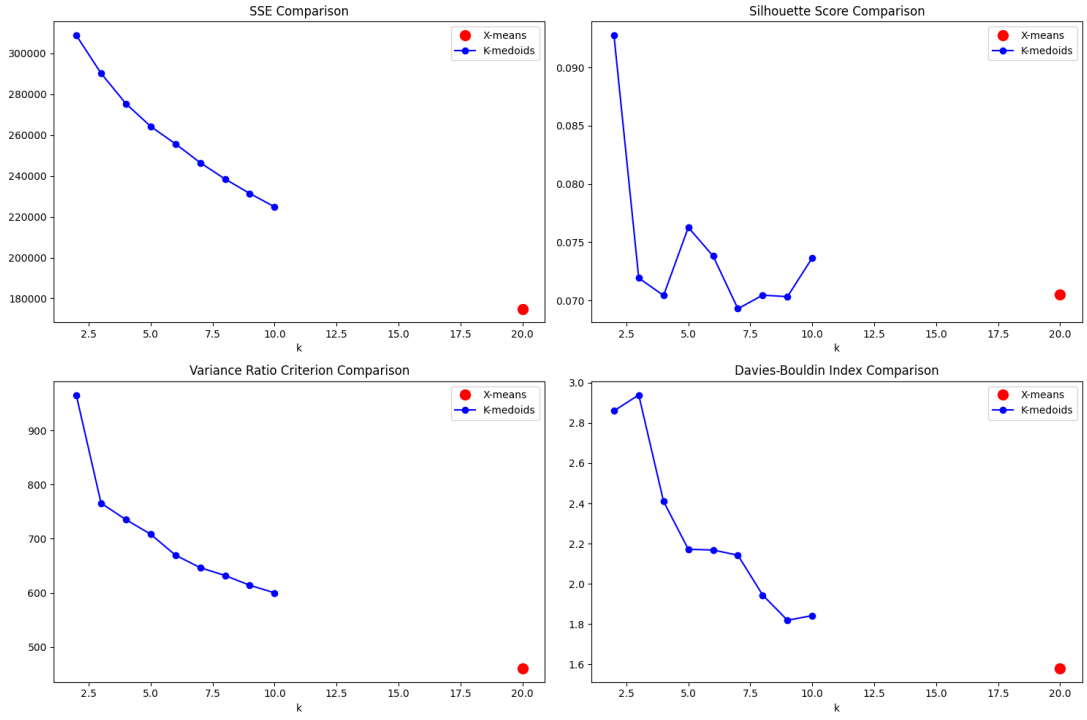

Figure 2.8: Results of running K-Medoids and X-Means clustering with different evaluation criteria.

Chapter 3

Predictive Analysis & XAI

3.1 Data Preparation

We loaded the cleaned datasets produced in the data-understanding phase, merged tracks with artist metadata, and defined the prediction target by mapping each artist region into a linguistically motivated macro-zone (rap school).

3.1.1 Macro–Zones Mapping

Macro-zones were defined to capture meaningful linguistic variation while maintaining sufficient samples per class.

Lombardia was mapped to a dedicated macro-zone, `Milan_Drill_School`, because it contributes a large share of tracks and is associated with a distinctive stylistic lexicon. Lazio was mapped to `Roman_Slang_School` for the same reason: high frequency in the dataset and strong regional slang cues. These single-region macro–zones reduce overlap with other regions and help maintain clearer class boundaries.

A third macro–zone, `South_Dialect_School`, groups southern and island regions where dialectal forms are frequently reflected in everyday language and lyrical expression.

Finally, the remaining regions (mostly north and central Italy) were grouped as `Lyrical_Standard_School` reflecting a closer proximity to standard Italian compared to strongly dialectal areas.

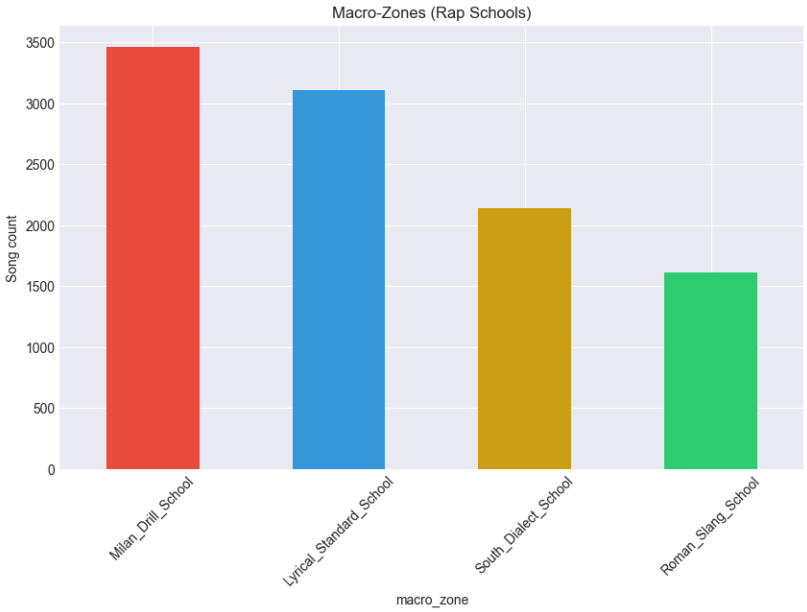

Figure 3.1: Distribution of Macro–Zone Schools

3.1.2 TF–IDF

To represent lyrical content, we applied TF–IDF (term frequency–inverse document frequency) to the `lyrics` column after handling missing lyrics (empty strings). To keep the representation computationally feasible, we limited the TF–IDF vocabulary to the top 2000 terms by corpus frequency. This representation highlights terms that are frequent within a track but less frequent across the full corpus, helping the classifier capture regional lexical cues (e.g., slang/dialect usage) rather than relying only on numeric audio attributes.

3.1.3 Feature Selection

After TF–IDF extraction, we removed leakage-prone and non-informative attributes (e.g., identifiers, names, free-text metadata, and geographic fields) to prevent the models from learning shortcuts unrelated to lyrical or audio characteristics. The target variable was the macro–zone label. We label-encoded the target classes, while the remaining input features were numeric. Finally, we created an 80/20 stratified train/test split to preserve class proportions.

3.2 Model Training and Hyperparameter Tuning

3.2.1 Grid Search and Cross-Validation

To select hyperparameters in a compute-efficient and reproducible way, we performed hyperparameter tuning using `GridSearchCV` with **3-fold Stratified Cross-Validation**. Stratification preserves the class distribution of the four macro-zone labels in each fold. The selection metric was **macro F1-score** (F1-macro), which weights all classes equally and is appropriate under class imbalance.

For each model, the total number of fitted models during tuning is:

$$\text{total fits} = (\text{ Number of candidates}) \times (\text{ Number ofCV folds})$$

All tuning was performed on the training split only; the held-out test set was used only for the final evaluation.

3.2.2 Random Forest

For Random Forest, we performed a grid search over **8** candidate configurations. With 3-fold cross-validation, this resulted in **24** total fits. The tuned hyperparameters included: `n_estimators`, `max_depth`, `min_samples_leaf`, `min_samples_split`, and `max_features`.

Hyperparameter	**Value**
n_estimators	250
max_depth	6
min_samples_leaf	10
min_samples_split	20
max_features	sqrt
Best CV F1-macro	**0.4899**

Table 3.1: Best Random Forest hyperparameters and corresponding 3-fold CV F1-macro score.

3.2.3 XGBoost

For XGBoost, we performed a grid search over **8** candidate configurations. With 3-fold cross-validation, this resulted in **24** total fits. The tuned hyperparameters included: `n_estimators`, `learning_rate`, `max_depth`, `min_child_weight`, `subsample`, `colsample_bytree`, `reg_alpha`, and `reg_lambda`.

Hyperparameter	Value
n_estimators	300
learning_rate	0.05
max_depth	3
min_child_weight	10
subsample	0.7
colsample_bytree	0.7
reg_alpha	10
reg_lambda	10
Best CV F1-macro	**0.4918**

Table 3.2: Best XGBoost hyperparameters and corresponding 3-fold CV F1-macro score.

3.2.4 LightGBM

For LightGBM, we performed a grid search over **8** candidate configurations. With 3-fold cross-validation, this resulted in **24** total fits. The tuned hyperparameters included: `n_estimators`, `learning_rate`, `num_leaves`, `min_child_samples`, `subsample`, `colsample_bytree`, and `reg_lambda`.

Hyperparameter	Value
n_estimators	500
learning_rate	0.05
num_leaves	15
min_child_samples	30
subsample	0.7
colsample_bytree	0.7
reg_lambda	50
Best CV F1-macro	**0.5794**

Table 3.3: Best LightGBM hyperparameters and corresponding 3-fold CV F1-macro score.

3.2.5 Support Vector Machine (Linear)

For the linear SVM, we performed a grid search over **3** candidate configurations. With 3-fold cross-validation, this resulted in **9** total fits. We used a pipeline consisting of feature standardization (`StandardScaler`) followed by `SVC` with a linear kernel. The tuned hyperparameters included: `C` and `class_weight`.

Hyperparameter	Value
kernel	linear
C	0.001
class_weight	balanced
Best CV F1-macro	**0.4798**

Table 3.4: Best linear SVM hyperparameters and corresponding 3-fold CV F1-macro score.

3.3 Model Evaluation

Hyperparameters were selected using **3-fold Stratified Cross-Validation** on the training split, optimizing **macro F1-score**. Final results were then reported on a held-out **test set** (80/20 split, stratified). The test set was not used during hyperparameter tuning.

For the boosting models (XGBoost and LightGBM), we applied **early stopping** using an internal validation split created from the training data (`X_train_es`/`X_val_es`), with patience set to 50 rounds. This prevents the test set from influencing training decisions while reducing overfitting.

3.3.1 Test Performance Comparison

Table 3.5 summarizes the models' performance on the held-out test set. LightGBM achieved the strongest baseline performance, and the refit experiment (training on the full training set using the early-stopped number of trees) provided an additional improvement.

Model	Train F1 (macro)	Test F1 (macro)	Train Acc	Test Acc	Train LogLoss	Test LogLoss
LightGBM_refit	0.8882	0.6040	0.8853	0.6089	0.5688	0.9477
LightGBM	0.8478	0.5966	0.8466	0.6035	0.6161	0.9651
XGBoost_refit	0.5849	0.5243	0.6142	0.5600	0.9971	1.0630
RandomForest	0.5479	0.5112	0.5670	0.5300	1.2807	1.2861
XGBoost	0.5749	0.5107	0.6043	0.5448	1.0138	1.0751
SVC	0.7779	0.5037	0.7731	0.5134	0.9247	1.1510

Table 3.5: Train/test performance comparison for all models.

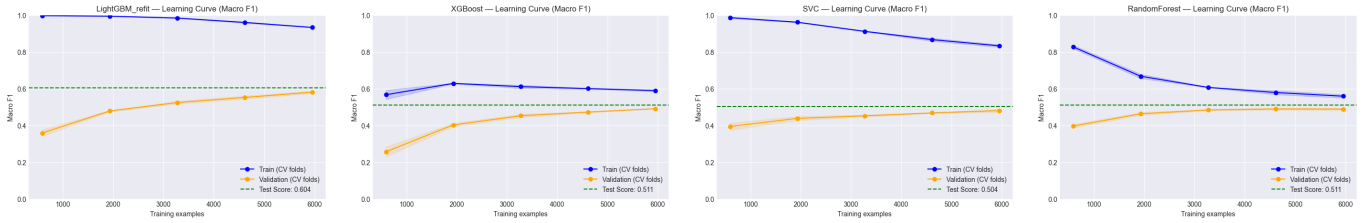

Figure 3.2: Learning curves (macro F1) for the compared models.

3.3.2 Early Stopping and Refit Experiment

Early stopping selected the number of boosting iterations using the internal validation split. We then ran an additional experiment where we **refit** the boosting models on the full training set using the selected number of trees.

Model	Early-stopped iteration	Refit n_estimators
XGBoost	299	300
LightGBM	500	500

Table 3.6: Early stopping iterations and refit configurations for boosting models.

The refit improved performance for both boosting models, with the largest gain observed for LightGBM:

- LightGBM: test macro F1 increased from 0.5966 to 0.6040 and test log loss decreased from 0.9651 to 0.9477.
- XGBoost: test macro F1 increased from 0.5107 to 0.5243 and test log loss decreased from 1.0751 to 1.0630.

3.3.3 Best Model Analysis

The best-performing model overall was **LightGBM_refit** (test macro F1 = 0.6040). The confusion matrix shows that most errors occur between linguistically closer rap schools (e.g., `Lyrical_Standard_School` vs `Milan_Drill_School`), suggesting overlap in vocabulary and style due to data imbalance. Despite these confusions, LightGBM maintained the strongest balance across all classes, as reflected by the highest macro F1-score.

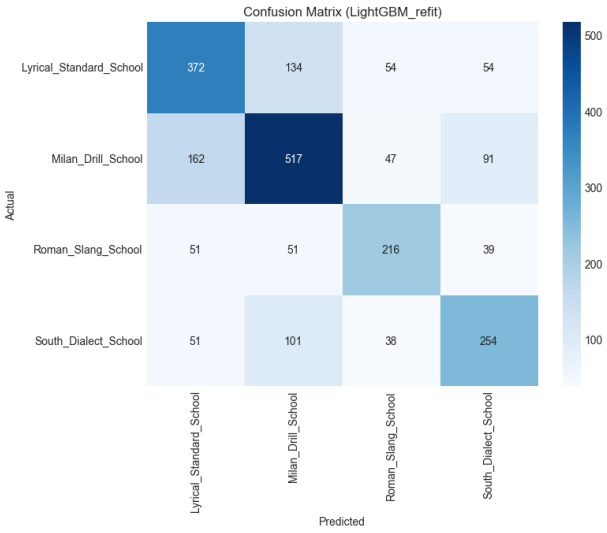

Figure 3.3: Confusion matrix of the best model (LightGBM_refit) on the test set.

3.4 Explainable AI with SHAP

To interpret the predictions of the best-performing classifier (LightGBM_refit in our final comparison), we used SHAP (SHapley Additive exPlanations). Because the feature space includes TF–IDF columns of the form `tfidf_i`, we first mapped each TF–IDF index back to its original token from the fitted TF–IDF vocabulary, so that SHAP explanations are readable.

3.4.1 Global explanations (feature importance)

Since LightGBM is a tree-based model, we used `shap.TreeExplainer` to compute SHAP values on the held-out test set. Figure 3.4 reports the most influential features according to mean absolute SHAP value, highlighting which audio/numerical attributes and which lyric tokens (TF–IDF words) contribute the most to separating the four rap-school classes.

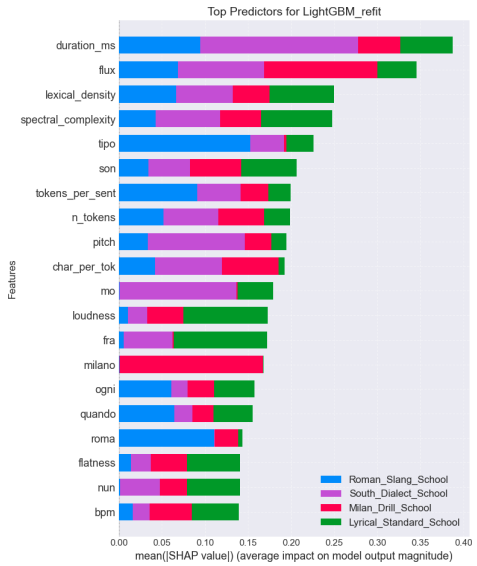

Figure 3.4: Global SHAP summary (bar plot) for the best model. Larger bars indicate higher mean absolute impact on the model output.

Qualitatively, we observe that both track-level numeric/audio descriptors (e.g., duration and spectral/loudness-related features) and region or slang-indicative lyric tokens can appear among the strongest signals, suggesting the model combines content and acoustic style to distinguish schools.

Chapter 4

Time series analysis

4.1 Preprocessing

We first load each song of the dataset using librosa. Then, leading and trailing sequences that are 25 or more decibels quiter than the rest of the song are trimmed. Next, the amplitude is normalized by peak amplitude. Beat tracking is used to calculate track tempo and divide into frames. MFCC, chroma and onset feature are extracted, where we end up with 13 MFCC, 12 chroma and 1 onset feature per frame.

4.2 Clustering

For clustering we decide to cluster aggregated variables instead of timeseries or subsequence clustering. We end up having a dataset with one feature vector for each song with 62 distinct features and no time dimension. The feature are then standardized by removing mean and scaling to unit variance. The dataset is clustered using K-Means with the same procedure as in Section 2.2, using K-Means++ a hundred times and evaluating different k from 2 to 10 using SSE, Sihouette Score, Variance Ratio Criterion and Davies-Bouldin Index. The results shown in Figure 4.1 indicate no

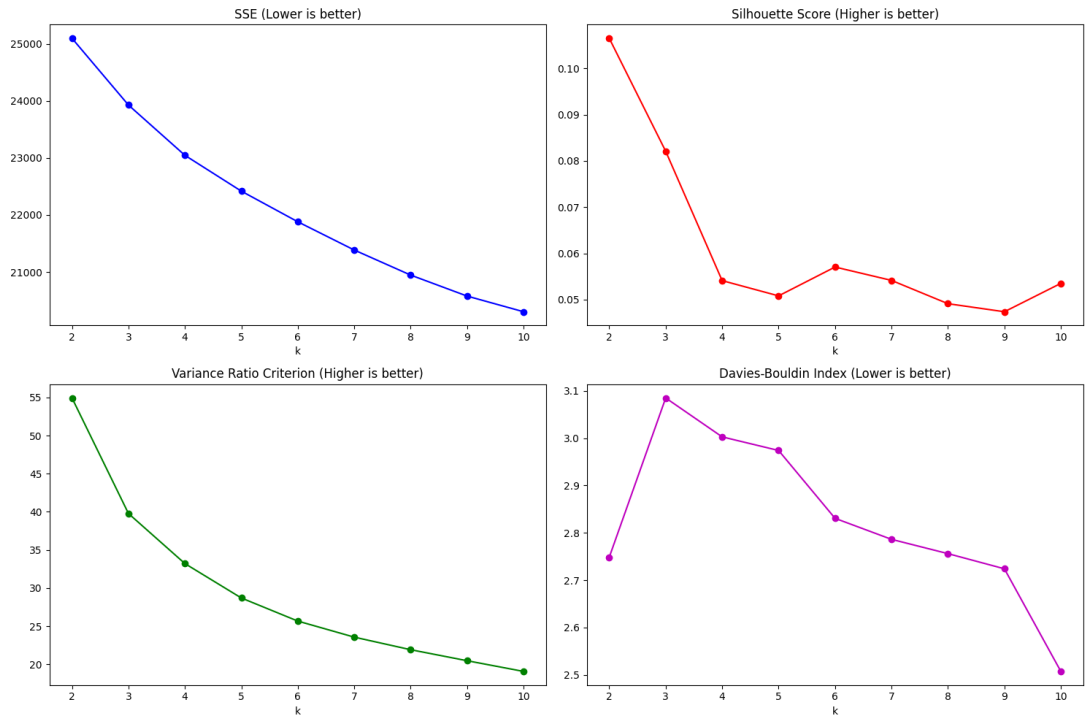

Figure 4.1: Results of running K-Means on the aggregated dataset with different evaluation criteria plotted for each value of k between 2 and 10.

good cluster separation with low Silhouette Score and Variance Ratio Criterion values, but we select $k = 2$ as the best possible clustering. Table 4.1 and Figure 4.2 also show poor clustering perfor-

Table 4.1: Clustering result of the most important features of K-Means clustering. Feature values are within-cluster averages.

Cluster ID	Members	mfcc_std_10	mfcc_std_09	mfcc_std_08	mfcc_std_06	mfcc_var_mean
Cluster 0	270	6.08	6.15	6.69	7.58	292.40
Cluster 1	184	7.82	8.18	9.12	10.51	582.90
All Clusters	454	6.79	6.97	7.68	8.77	410.14

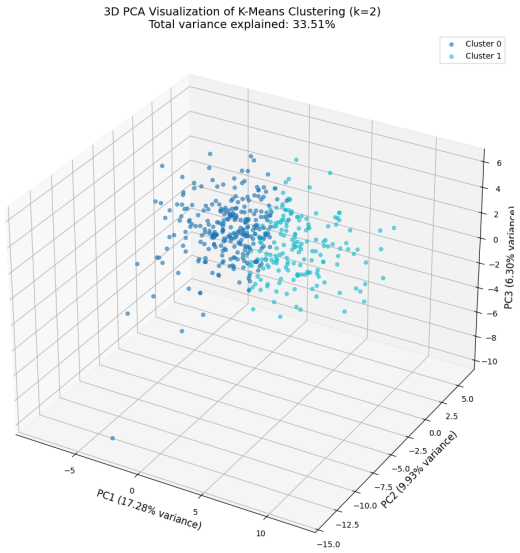

Figure 4.2: First three principal components of the dataset. Colors indicate cluster assignments of K-Means.

mance. 56% of songs in Cluster 0 are from Fabri Fibra and 67% in Cluster 1. In total, 61% are from Fabri Fibra, so the difference is too little to make room for any interpretations.

4.3 Motif Extraction

To reduce the timeseries dimensionality for motif extraction, we solely choose the chroma feature, which is twelve-dimensional, and reduce it further to one dimension by using entropy. The motifs we look for are 32 beats long. We extract no more than three motifs per song. Combining the motif analysis with clustering, we can calculate the average distance of the best motifs (meaning distance from each motif to its nearest neighbour) of the two clusters. Cluster 0 has a average best motif distance of 2.38, while it is 1.96 for Cluster 1. The median is 2.20 and 1.77 for Cluster 0 and Cluster 1 respectively. Lower motif distance means stronger repetition.

4.4 Anomaly Detection

Since we already have obtained cluster labels for the song timeseries, it is straightforward to use them for anomly detection. We use euclidean distance between the cluster average and each song in the cluster as a measure for detecting anomality. To visualize the data, we resample each timeseries to 128 datapoints. We use the 13 MFCC features and standardize the data. Figure 4.3 shows the median anomaly intensity for the clusters at each point in the resampled timeseries. There is no trend we can identify between the clusters, other than less anomalies at the beginning and end of the songs.

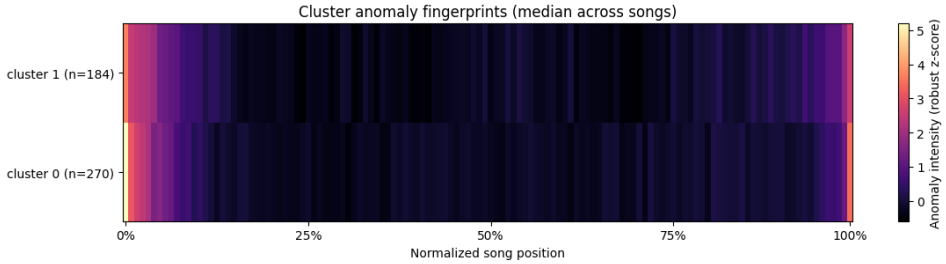

Figure 4.3: Median of anomaly intensity for the clusters at each point in the resampled song.

4.5 Shapelets

We train a shapelet classifier and use the authorship as the class label (i.e. Fedez or Fabri Fibra). Because of the volume of the data we use only the 13 MFCC features, resample to 128 datapoints and train only two short shapelets with length 16 and one larger shapelet with length 32, in 50 epochs. The resulting accuracy is 42% and F1-score is 0.41. If we match the shapelets to their most probable location in the timeseries, we can see which shapelets are more probable to belong to a certain artist. Table 4.2 shows the distribution of the artists for the ten best matching songs for each shapelet. The shapelets are plotted in 4.4.

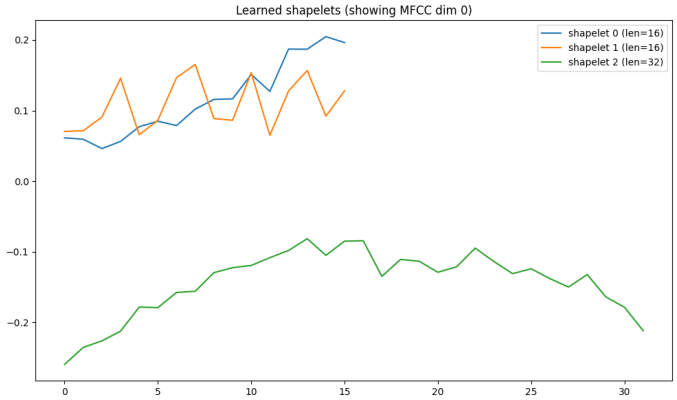

Figure 4.4: The three learned shapelets for the first timeseries in the dataset. Only the first MFCC dimension is plotted.

Table 4.2: Artists of the ten best matching songs for each shapelet.

	artist	ART07024718	ART25707984	dominance
shapelet_id	shapelet_len			
0	16	2	8	0.8
1	16	2	8	0.8
2	32	2	8	0.8

Chapter 5

Ethical and legal implications

The development of data analysis and predictive models for classifying Italian rap artists into geographical macro-zones introduces significant ethical and legal complexities. While this project aims to explore cultural trends through data analytics, the collection of personal artist data and the development (and eventual deployment) of classification algorithms raise concerns regarding privacy, intellectual property, and algorithmic fairness.

5.1 Ethical Implications

The development of data analysis and predictive models for classifying Italian rap artists into geographical macro-zones raises several ethical concerns related to privacy, representation, cultural identity, and algorithmic bias. While the project aims to explore cultural trends through data analytics, the act of profiling individuals and associating artistic expression with regional identities carries inherent risks.

5.1.1 Cultural Complexity Reduction and Stereotyping

A key ethical risk lies in reducing complex cultural identities to statistical patterns. By correlating features such as explicit language frequency, with specific regions (e.g., Lombardia versus Southern Italy), the model risks reinforcing existing regional stereotypes. For example, if "Southern Rap" becomes statistically associated with specific slang, such representations could perpetuate misinterpretations of Southern Italian culture.

Artists whose music deviates from these learned patterns such as Southern artists producing softer rap may be rendered algorithmically invisible or misclassified. This reduction of rich, diverse cultural traditions into binary labels risks offending the communities being modeled.

5.1.2 Privacy and Purpose Limitation

This project processes personal attributes of identifiable artists, including gender, birth year, and birthplace. Although this information is largely publicly available (e.g., Wikidata and Wikipedia), artists did not provide explicit consent for their data to be repurposed for predictive profiling. Reclassifying individuals into regional categories they may not identify with introduces ethical tension with the principle of purpose limitation, as the original intent of data publication was informational rather than analytical or classificatory.

5.1.3 Gender Bias and Representation

The dataset is predominantly male, reflecting broader gender imbalances within the Italian rap scene. Consequently, the learned regional "signatures" are heavily influenced by male linguistic patterns and thematic content. Female artists, who may express different topics or stylistic choices, are likely

to be misclassified or treated as statistical outliers, further reinforcing their under-representation and marginalization within algorithmic systems.

5.2 Legal Implications

5.2.1 Data Protection and GDPR Compliance

Despite relying on publicly available sources, the processing of artist data falls within the scope of the General Data Protection Regulation (GDPR), as it involves identifiable natural persons. Profiling individuals based on inferred characteristics such as regional affiliation derived from lyrical content may constitute automated profiling under GDPR, requiring clear justification, transparency, and appropriate safeguards.

Although anonymisation of artists is generally considered a best practice under data protection principles, it was intentionally not applied in this study to preserve interpretability. Anonymising the data would have limited our ability to assess whether the identified patterns were meaningful and contextually valid, a trade-off addressed through restricted use within an academic context.

5.2.2 Copyright and Text and Data Mining

The use of song lyrics for Natural Language Processing introduces copyright considerations. The EU Directive on Copyright in the Digital Single Market (Directive (EU) 2019/790[4]) provides exceptions for text and data mining (Article 3), particularly for scientific research conducted by research institutions, which applies to this university project. However, these exceptions are limited in scope. If the project were to be commercialized or deployed beyond academic research, training models on full song lyrics to extract stylistic patterns could infringe upon artists' intellectual property rights, as it would derive value from copyrighted material without direct authorization or compensation.

5.3 Mitigation Strategies

To address the identified risks, the following strategies must be integrated into the workflow:

- **Fairness-Aware Modeling**

We must move beyond "F1-Score" and evaluate the model using fairness metrics (e.g., Disparate Impact or Equalized Odds). We should verify if the model performs equally well for female artists as it does for male artists, and identifying if specific regions have higher false-positive rates due to stereotyping.

- **Explainable AI (XAI)**

Implementing SHapley Additive exPlanations (SHAP) is non negotiable. We must inspect why the model classifies an artist as "South." If the top features are disrespectful terms or stereotypes, the model must be retrained or those features removed to prevent harm.

- **Human-in-the-Loop Validation**

The model's predictions should never be used for automated decision-making (e.g., rejecting an artist from a playlist) without human review. Subject matter experts should validate that the "regional signatures" identified by the model are culturally accurate and not merely artifacts of bias in the training data.

Bibliography

[1] David Arthur and Sergei Vassilvitskii. k-means++: the advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA '07, page 1027–1035, USA, 2007. Society for Industrial and Applied Mathematics.

[2] T. Caliński and J Harabasz. A dendrite method for cluster analysis. *Communications in Statistics*, 3(1):1–27, 1974.

[3] David L. Davies and Donald W. Bouldin. A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2):224–227, 1979.

[4] European Parliament and Council of the European Union. Directive (eu) 2019/790 of the european parliament and of the council of 17 april 2019 on copyright and related rights in the digital single market and amending directives 96/9/ec and 2001/29/ec. Official Journal of the European Union, L 130, May 2019. Text with EEA relevance.

[5] Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987.

[6] Erich Schubert. Stop using the elbow criterion for k-means and how to choose the number of clusters instead. *ACM SIGKDD Explorations Newsletter*, 25(1):36–42, June 2023.

[7] Gideon Schwarz. Estimating the dimension of a model. *The Annals of Statistics*, 6(2):461–464, 1978.

[8] Robert L. Thorndike. Who belongs in the family? *Psychometrika*, 18:267–276, 1953.

Chapter 6

Appendix

6.0.1 Local explanations (single prediction)

To provide an example at the instance level, we also generated a SHAP waterfall plot for a randomly selected test track, showing the top features pushing the model toward the predicted class and away from the alternatives.

example:

Song ID: TR475477 Artist: Dani Faiv Song Name: Ultimo piano Region: Liguria Predicted Macrozone: Lyrical_Standard_School Actual Macrozone: Lyrical_Standard_School

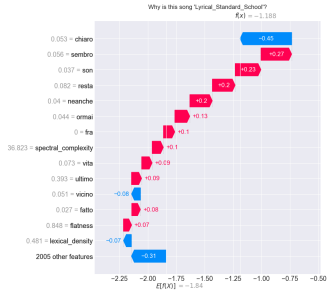

Figure 6.1: Local SHAP summary showing the attributes the model used to decide the prediction.